

BIM492 – Design Patterns

Term Project: Library Management System

Design Patterns Report

Group 29

Gökay Emre Duymuş | Ekrem Geniş

This report describes the three design patterns used in the implementation of the Library Management System and explains the reasoning behind each choice. The patterns are not applied for structural decoration; each one addresses a concrete problem that arose during design and directly changes how the system behaves at runtime.

1. Command Pattern

1.1 Motivation

The modification module requires the ability to undo the most recent change to a book record. The initial approach of saving a copy of the modified fields before each update quickly becomes unmanageable, as the responsibility for tracking previous states would be spread across multiple methods. A more structured solution was needed that could encapsulate both the change and its reversal in a single unit.

1.2 Application

The pattern is implemented through three classes. **Command** is an interface that declares two methods: *execute()* and *undo()*. **ModifyBookCommand** implements this interface and represents a single modification operation. When constructed, it immediately takes a deep copy of the book's current state before any changes are applied. This copy serves as the restore point if the user later requests an undo.

```
public ModifyBookCommand(Book book, Book afterState) {
    this.beforeState = new Book(book); // deep copy of current
    state
    this.afterState = afterState;
}

@Override
public void undo() {
    applyState(book, beforeState);
}
```

CommandHistory manages a stack of executed commands using a *Deque<Command>*. Each call to *execute(cmd)* runs the command and pushes it onto the stack. When the user selects the undo option from the modify menu, *CommandHistory.undo()* pops the most

recent command and calls its *undo()* method. The user interface has no direct involvement in the reversal logic; it simply delegates to *LibraryService.undoLastModification()*.

2. Observer Pattern

2.1 Motivation

When a book is borrowed or returned, the system must update two pieces of state: the availability flag and the borrow count. One option is to handle both updates directly inside *LibraryService*. However, this approach ties unrelated concerns together and makes it harder to extend the borrowing behavior in the future. The Observer pattern was chosen to keep the notification logic separate from the service that triggers it.

2.2 Application

BookObserver is an interface that defines two methods: *onBookBorrowed(Book)* and *onBookReturned(Book)*. **AvailabilityTracker** is the concrete implementation. When notified of a borrow event, it sets the book's available field to false and increments the borrow counter. When notified of a return, it sets available back to true.

LibraryService acts as the subject. It maintains a *List<BookObserver>* and iterates over it whenever a borrow or return action takes place:

```
for (BookObserver observer : observers) {  
    observer.onBookBorrowed(book);  
}
```

AvailabilityTracker is registered in the *LibraryService* constructor. Since observers are referenced through the interface, additional behaviors such as logging or member notifications can be introduced by implementing *BookObserver* and registering the new instance, without modifying the existing service or tracker code.

3. Strategy Pattern

3.1 Motivation

Search results must be sortable in ascending or descending order by title, and the choice is made by the user at runtime. Handling this with a conditional inside the search method works for two options but does not scale well. If a new sort criterion were added later, the method would need to be modified directly. The Strategy pattern makes it possible to change the sorting behavior at runtime without touching the search logic.

3.2 Application

SortStrategy is an interface with a single method: *sort(List<Book>)*. **AscendingTitleSort** and **DescendingTitleSort** each implement this interface, sorting the given list alphabetically in the respective direction using a case-insensitive comparator.

LibraryService holds a *SortStrategy* field. After the user enters a search keyword, the interface asks for a sort preference and calls either *setSortAscending()* or *setSortDescending()* on the service. The actual sorting is then delegated entirely to the active strategy:

```
public void applySorting(List<Book> results) {  
    sortStrategy.sort(results);  
}
```

The service itself contains no sorting logic. Swapping the strategy object at runtime is sufficient to change the behavior, and adding a new sort order in the future requires only a new class that implements *SortStrategy*.